

从 3D 高斯到三角网格：SuGaR 复现 与深度-法向一致性正则改动

浙江大学博士代码考核 | 6 小时限时复现与魔改 | 全中文技术报告

目标开源项目： SuGaR (CVPR 2024) , 官方 commit 7c10c4a

数据集与场景： Tanks & Temples / Truck (3DGS 官方打包 tandt_db.zip, 含 COLMAP 稀疏重建)

改动点： ① 训练端：在 coarse SuGaR 训练主循环中加入深度-法向一致性正则 L_DNC (借鉴 2DGS 思路, 自行实现) ; ② 提取端：把泊松重建深度改为按场景尺度自动推算 (移植自 Frosting)

对照组： 训练端五组完整重跑： $\lambda=0 / 0.05 / 0.2 / 0.2+\text{detach}$ 诊断 / 官方 dn_consistency; 提取端在同一个 $\lambda=0$ 模型上扫 Poisson 深度与顶点密度分位

报告生成时间： 2026 年 09 月 13 日 09:59

数据来源声明

本报告中出现的全部实验数值均由脚本从 outputs/metrics/summary.csv 及 outputs/metrics/ 下的 render_*.json、geometry_*.json、各 run 的 train_stats.json 与 dnc_log.csv 自动读取并排版, 未经人工转录; 未跑出结果的位置显示“未完成”, 不做任何估计或补值。

CUDA 光栅化 (diff-gaussian-rasterization) 的原子累加本身非确定, 即使固定 seed=0, 逐次运行也会有微小数值差异, 本报告中的指标据此理解。

摘要

本报告完成了 SuGaR (Surface-Aligned Gaussian Splatting) 从 3D 高斯到三角网格的完整短流水线复现：3D Gaussian Splatting 训练 7000 迭代 $\rightarrow$ coarse SuGaR 密度正则训练至 15000 迭代 $\rightarrow$ 泊松表面重建导出三角网格，并在此基础上做了一处进入训练主循环的改动：深度-法向一致性正则 (Depth-Normal Consistency, 下称 DNC)。改动思路借鉴 2D Gaussian Splatting

(SIGGRAPH 2024) 提出的 depth-normal consistency 损失，将其移植到 SuGaR 的三维椭球表示上，并针对法向符号歧义、低不透明度像素与物体边缘设计了掩码。实验在 Tanks & Temples 的 Truck 场景上以 $\lambda \in \{0, 0.05, 0.2\}$ 做三组完整对照，统一评测渲染质量 (PSNR / SSIM / LPIPS) 与网格几何 (稀疏点到网格距离、拓扑碎片、法向平滑度)。

本工作在 Tanks&Temples Truck 场景上完整复现了 3DGS(7k) $\rightarrow$ coarse SuGaR(15k, density 正则) $\rightarrow$ Poisson 网格提取的短流程，并在 coarse 训练主循环中加入像素级的深度-法向一致性正则 (DNC)。对照实验 ($\lambda=0 / 0.05 / 0.2$, 各一次完整重跑) 表明：在 7k $\rightarrow$ 15k 的短预算下，DNC 没有带来几何改善，反而随 λ 增大单调损害渲染与几何； $\lambda=0.2$ 时场景坍缩为大尺度半透明薄片。进一步的诊断实验 (切断损失对深度的梯度) 把 PSNR 从 13.24 dB 恢复到 24.40 dB ($\lambda=0$ 为 24.62 dB)，证实退化源于“抹平渲染深度”这一平凡解，但几何指标仍未优于基线。本报告如实报告这一负结果与其机理，在写作后期核实到官方仓库已内置同类正则 (dn_consistency) 并将其作为对照组：官方实现在同样 $\lambda=0.05$ 下碎片分量 -16%、二面角均值 -10%、PSNR 仅 -0.24 dB。因此结论修正为：深度-法向一致性这一思路在 SuGaR coarse 阶段有效，本工作独立实现的版本因实现细节差异而失效；本报告如实给出负结果、退化机理与两版实现的差异清单。

预注册的可检验假设

H1 (几何)： $\lambda>0$ 时，网格碎片连通分量数下降、COLMAP 稀疏点到网格距离中位数下降、网格法向更平滑。

H2 (渲染代价)： 测试视角 PSNR 变化在 ± 0.5 dB 以内；若下降更多，如实报告为代价。

H3 (叠加关系)： DNC 与 SuGaR 原有的 SDF 法向损失不冲突，训练中两者同时下降。

- H1（几何改善）：**不支持**。 $\lambda=0.05$ 时碎片分量 2137 $\rightarrow$ 2324、稀疏点到网格距离中位数 0.0687% $\rightarrow$ 0.1421%（相对相机尺度）、二面角均值上升； $\lambda=0.2$ 的碎片数下降到 1177 是网格整体坍塌（面数 370k $\rightarrow$ 223k，包围盒从 ± 23 缩到 ± 9.7 ）的副产物，不构成证据；detach 组碎片 2142、距离中位数 0.0839%，仍不优于 $\lambda=0$ 。
- H2（渲染代价 ± 0.5 dB 内）：**不支持**。 $\lambda=0.05$ 为 -2.17 dB， $\lambda=0.2$ 为 -11.38 dB。仅 detach 变体满足 (-0.22 dB)。
- H3（与原有 SDF 法向损失不冲突）：**仅在"两者同时下降"的操作性定义下成立**；两组 λ 的 L_{dnc} 与 $\text{sdf_better_normal_loss}$ 均单调下降，但几何并未变好，说明"损失下降"与"表面质量"在此脱钩，H3 的定义本身不足以作为质量判据。

1 环境与数据

1.1 运行环境

表 1 运行环境与关键依赖版本

项	值
操作系统	Ubuntu 22.04.3 LTS
GPU	2 $\times$ NVIDIA H200 (143771 MiB / 卡，驱动 575.57.08)
python	3.10.12 (main, Jan 8 2026, 06:52:19) [GCC 11.4.0]
venv	/scratch/e1351071/virtualenvs/zju_test
torch	2.4.1+cu121
torchvision	0.19.1+cu121
torch.version.cuda	12.1
cuda available	True device count: 2
GLIBCXX_USE_CXX11_ABI	False
matplotlib	3.10.9
numpy	1.26.4
open3d	0.19.0
opencv-python	4.11.0.86
plyfile	1.1.3

项	值
pytorch3d	0.7.8
scikit-learn	1.7.2
scipy	1.15.3
simple-knn	0.0.0
commit	7c10c4ae4a267dece512f5c7f40ed212a0a2ab44
系统 nvcc	12.1

来源: notes/ENV.md (由 scripts/gen_env_md.sh 自动抓取), 完整冻结清单见 notes/pip_freeze.txt。

1.2 数据集与划分

表 2 数据集、分辨率与训练/测试划分

项	值
数据来源	3DGS 官方发布包 tandt_db.zip (含 Tanks & Temples 与 Deep Blending)
使用场景	data/tandt/truck (含 COLMAP sparse/0 稀疏重建)
图像总数	251 张
渲染分辨率	979 × 546 像素
训练 / 测试划分	lffhold=8 (按文件名排序后索引 % 8 == 0 为测试) → 训练 219 张 / 测试 32 张
几何参考	COLMAP 稀疏点 (track 长度 ≥ 3 且落在前景包围盒内), 参考点数 88,066 个

所有 run (含 3DGS、三组 coarse SuGaR 与网格提取) 使用完全相同的划分与相机。

2 复现流程与命令

完整流水线分三步, 三组对照共用同一个 3DGS 检查点以保证公平:

- 第一步: 3D Gaussian Splatting 训练 7000 迭代, 产出
point_cloud/iteration_7000/point_cloud.ply;
- 第二步: coarse SuGaR (density 正则模式) 从该检查点续训到 15000 迭代, 其中 7000–9000 为熵正则阶段, 9000 迭代做一次 opacity<0.5 的硬剪枝, 9000 起进入 SDF 正则;

- 第三步：extract_mesh.py 以 surface_level=0.3、decimation 目标 200000 做泊松重建与抽取。

关于 decimation 语义的勘误：命令行 -d 200000 并非“输出 20 万面”，而是对前景网格与背景网格各自抽取到 20 万个三角形的目标，两者合并后总面数约为 37 万，属正常现象。三组使用完全相同的参数。

第一步（3DGS 7000 迭代）的实际命令：

```
cd $PROJ_ROOT/repo/SuGaR
CUDA_VISIBLE_DEVICES=0 python gaussian_splatting/train.py \
-s $PROJ_ROOT/data/tandt/truck \
-m $PROJ_ROOT/outputs/baseline/gs_truck \
--iterations 7000 --save_iterations 7000 --test_iterations 7000 --eval
```

第二、三步（三组 coarse 训练 + 网格提取）的实际命令：

```
cd /scratch/e1351071/zju_test/repo/SuGaR_dev

# --- coarse_base_seed0 (GPU0,  $\lambda=0$ ) ---
python train_coarse_density.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ \
-i 7000 -o /scratch/e1351071/zju_test/outputs/runs/coarse_base_seed0 \
--eval True --gpu 0 --seed 0 --dnc_factor 0 --dnc_start 9000
python extract_mesh.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ -i 7000 \
-m
/scratch/e1351071/zju_test/outputs/runs/coarse_base_seed0/sugarcoarse_3Dgs7000_densityestim02_sdfnorm02/15000.pt \
-l 0.3 -d 200000 --eval True --gpu 0 \
-o /scratch/e1351071/zju_test/outputs/runs/mesh_base_seed0

# --- coarse_dnc005 (GPU1,  $\lambda=0.05$ ) ---
python train_coarse_density.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ \
-i 7000 -o /scratch/e1351071/zju_test/outputs/runs/coarse_dnc005 \
--eval True --gpu 1 --seed 0 --dnc_factor 0.05 --dnc_start 9000
python extract_mesh.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ -i 7000 \
-m
/scratch/e1351071/zju_test/outputs/runs/coarse_dnc005/sugarcoarse_3Dgs7000_densityestim02_sdfnorm02/15000.pt \
-l 0.3 -d 200000 --eval True --gpu 1 \
-o /scratch/e1351071/zju_test/outputs/runs/mesh_dnc005

# --- coarse_dnc02 (GPU1,  $\lambda=0.2$ ) ---
python train_coarse_density.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ \
-i 7000 -o /scratch/e1351071/zju_test/outputs/runs/coarse_dnc02 \
--eval True --gpu 1 --seed 0 --dnc_factor 0.2 --dnc_start 9000
python extract_mesh.py \
```

```
-s /scratch/e1351071/zju_test/data/tandt/truck \  
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ -i 7000 \  
-m  
/scratch/e1351071/zju_test/outputs/runs/coarse_dnc02/sugarcoarse_3Dgs7000_densityestim02_sdfn  
orm02/15000.pt \  
-l 0.3 -d 200000 --eval True --gpu 1 \  
-o /scratch/e1351071/zju_test/outputs/runs/mesh_dnc02
```

以上命令原样摘自 notes/RUNLOG.md 与 notes/RUNLOG_s34.md，未做改写；完整命令另见附录 A。

随机性控制：3DGS 的 `safe_state()` 固定 `random / numpy / torch` 的 `seed` 为 0；三组 coarse 训练显式传入 `--seed 0`。但 CUDA 光栅化的原子累加本身非确定，因此逐次运行仍会有微小数值差异，本报告不把 0.01 dB 量级的差异当作显著差异。

3 改动动机与实现

本工作一共做了两处进入主流程的改动：训练端的深度-法向一致性正则（本章，是主要改动），以及提取端的自动泊松深度（移植自 Frosting，见 5.4 节）。两者互不依赖，可以单独开关。

3.1 动机

SuGaR 用 SDF / density 正则把高斯压扁并贴到表面上，但每个高斯的法向只受到采样点级别的 SDF 约束，相邻高斯之间的法向可以互相不一致。泊松重建以表面采样点及其法向为输入，法向噪声会直接转化为网格褶皱与漂浮碎片。DNC 的想法是：既然渲染深度图本身就编码了几何，那么由深度图差分得到的几何法向 N_d 与渲染出的高斯法向 N 应当一致；在像素尺度上强制两者一致，等价于要求相邻高斯共面。

3.2 思路来源与自己的贡献

思路来源：2D Gaussian Splatting (SIGGRAPH 2024, 文献 [3]) 提出的 depth-normal consistency 损失。本工作把它移植到 SuGaR 的 coarse 阶段，属于借鉴已发表思路并自行实现，未复制 2DGS 的任何代码。相对原思路，本实现需要额外处理三件事：

- P1 表示差异：2DGS 的基元是二维圆盘，法向是其固有属性；SuGaR 的基元是三维椭球，法向取最短 scale 轴经四元数旋转后的方向，且存在正负两个朝向，需要按相机方向翻转并在损失中取绝对值；

- P2 与原有损失的叠加关系：SuGaR 已有采样点级别的 SDF 法向损失，DNC 是像素级、跨高斯的约束，两者是否冗余或冲突需要用实验回答（对应假设 H3，见图中训练曲线）；
- P3 掩码设计：渲染法向图是 α 合成的结果，低不透明度像素会得到接近零向量、方向无意义的法向，因此在计划原有的三条掩码之外增加了 $\|N_{\text{raw}}\| > 0.1$ 的数值保护条件。

3.3 损失定义

$$P(u, v) = \text{unproject}(x_{\text{ndc}}(u), y_{\text{ndc}}(v), D(u, v))$$

① 用相机内参把渲染深度 D 反投影成相机系点图 P

$$N_d = \text{normalize}\left(\frac{\partial P}{\partial u} \times \frac{\partial P}{\partial v}\right)$$

② 对点图做中心差分并叉乘，得到几何法向 N_d （统一翻转为朝向相机）

$$M = \{(u, v) : D < 0.98 D_{\text{max}} \wedge |\nabla D| < 0.05 D \wedge |N_{\text{raw}}| > 0.1 \wedge \text{border}\}$$

③ 有效像素掩码：剔除背景、深度跳变、低不透明度像素与图像边缘 2 像素

$$\mathcal{L}_{\text{DNC}} = \frac{1}{|M|} \sum_{(u, v) \in M} (1 - |N(u, v) \cdot N_d(u, v)|)$$

④ 掩码内像素的平均法向不一致度；取绝对值以消除椭球最短轴的法向符号歧义

$$\mathcal{L} = \mathcal{L}_{\text{SuGaR}} + \lambda \mathcal{L}_{\text{DNC}}$$

⑤ 与 SuGaR 原有损失相加； $\lambda=0$ 时完全不进入 DNC 分支（代码路径与原版一致）
 N =渲染法向图（高斯最短轴法向经 α 合成）； D =渲染深度图； λ =--dnc_factor

图 1 深度-法向一致性正则 $\mathcal{L}_{\text{DNC}}$ 的定义与有效像素掩码

实现要点： $\lambda = 0$ 时完全不进入 DNC 分支（不渲染额外的深度图与法向图），保证基线的代码路径与原版 SuGaR 完全一致；DNC 从第 9000 迭代起生效，与 SuGaR 自身的 SDF 正则同期；每 100 迭代把 $\mathcal{L}_{\text{DNC}}$ 、有效像素占比与两项 SDF 损失写入 dnc_log.csv，每 1000 迭代把 $D / N / N_d /$ 掩码 四张图落盘。

3.4 代码改动范围

表 3 改动涉及的文件与行数统计

文件	新增行	删除行
sugar_trainers/coarse_density.py	225	1
sugar_utils/dnc_utils.py	394	0
train_coarse_density.py	31	0

来源：notes/dnc_stage3.patch（git diff 留痕）。合计新增 650 行、删除 1 行；改动只涉及训练入口参数、coarse 训练循环与一个新增的独立工具模块，未触碰网格提取与评测代码。

4 实验设置与对照

表 4 三组对照的配置差异（其余超参完全一致）

组别	run 目录名	λ (--dnc_factor)	DNC 起始迭代	输出目录
基线 $\lambda=0$	coarse_base_seed0	0	—	outputs/runs/ coarse_base_seed0
本工作 $\lambda=0.05$	coarse_dnc005	0.05	9000	outputs/runs/ coarse_dnc005
本工作 $\lambda=0.2$	coarse_dnc02	0.2	9000	outputs/runs/ coarse_dnc02
$\lambda=0.2+\text{detach}$	coarse_dnc02_detach	0.2	9000	outputs/runs/ coarse_dnc02_detach
官方 DNC	coarse_official_dnc	0.05	9000	outputs/runs/ coarse_official_dnc
3DGS 7k(参考)	vanilla3dgs7k	—	—	未完成
基线(阶段 2 留档)	coarse_baseline	—	—	outputs/runs/ coarse_baseline

共同固定量：同一 3DGS 7000 迭代检查点、seed=0、总迭代 15000、estimation_factor=0.2、normal_factor=0.2、网格提取 surface_level=0.3 与 decimation 200000、同一 lloffhold=8 划分。

留档说明：阶段 2 中在未打补丁的原版代码上跑过一次基线（run 名 coarse_baseline），该结果仅作为“补丁未改变 $\lambda=0$ 代码路径”的旁证留档，不参与三组正式对照；正式对照的三组统一使用打补丁后的代码与 seed=0。

阶段 2 用官方原版代码跑的 coarse_baseline（无 seed、无插桩）与打补丁后 $\lambda=0$ 的 coarse_base_seed0 相比：PSNR 24.653 vs 24.620，SSIM 0.8564 vs 0.8555，距离中位数 0.0684% vs 0.0687%，连通分量 2237 vs 2233，碎片 2147 vs 2137，高斯数 429,075 vs 429,423。差异均在 CUDA 光栅化非确定性与随机采样带来的抖动量级内，据此判断 $\lambda=0$ 时补丁没有改变原有代码路径（日志中无任何 DNC 分支输出）。

4.1 过程记录：首轮 λ 组被中断与重跑

本次考核在计算集群的交互式作业中进行。首轮三组对照采用双卡并行调度（ $\lambda=0$ 独占一卡，两个 $\lambda>0$ 组共享另一卡），但两个 $\lambda>0$ 组的训练在接近收尾时随作业的 walltime 到期被强制终止，未能写出最终检查点。恢复作业后只分配到单张 GPU，因此改为顺序重跑两个 $\lambda>0$ 组：先 $\lambda=0.2$ ，再 $\lambda=0.05$ ，其余超参、种子、数据划分与网格提取参数与首轮完全一致。 $\lambda=0$ 组在首轮已完整跑完训练与网格提取，未受影响，不需要重跑。

按照考核要求保留失败记录，首轮被中断的两个目录原样保留（仅重命名以避免与重跑结果混淆），其中的逐迭代日志与 DNC 中间量快照仍可作为“训练确实进行到了什么程度”的证据：

表 5 首轮被中断的 λ 组残留记录（保留为失败证据，不参与正式对照）

残留目录	dnc_log 最后迭代	日志行数	中间量快照数	最后快照迭代	是否有 15000.pt
coarse_dnc005_killed_0649	14900	59	5	14000	否（被中断，无最终检查点）
coarse_dnc02_killed_0649	14900	59	5	14000	否（被中断，无最终检查点）

表中数字由脚本直接从磁盘上的残留文件统计得到（dnc_log.csv 的最大迭代号、dnc_vis/ 下 iter*_depth.png 的数量与最大迭代号），不是人工记录。这些目录只用于说明中断发生的时点，其指标不进入任何结论。

06:49 首个 PBS 作业 walltime 到期被强杀，此时 $\lambda=0.05$ 与 $\lambda=0.2$ 两组均已训练到 14900/15000 迭代；由于 SuGaR 不保存中间检查点，两组全部重跑。07:57 起在新作业（hopper-13，仅 1 张 H200）上顺序重跑： $\lambda=0.2$ 于 08:18 完成训练与提取， $\lambda=0.05$ 于 08:33 完成训练；其网格提取因运行中的脚本被并发编辑而中断，08:34–08:40 手动补跑。诊断组 $\lambda=0.2+\text{detach}$ 于 08:40–09:00 独占 GPU 完成。首轮被杀的两组残留目录与日志以 *_killed_0649 后缀保留，首轮 $\lambda=0.2$ 在 14900 迭代时已呈现与重跑组一致的坍塌形态，说明该现象可复现，非偶然。由于重跑发生在不同节点、且 OMP 线程数在组间不一致（ $\lambda=0$ 组为 24，其余为 12），训练代价列只能作量级参考，不是受控对照。

写作后期追加的两组实验的时间线（逐字取自各阶段 SELF_CHECK 的日志表）：① 官方

dn_consistency 对照组 —— 等待脚本 08:34:34 进入 GPU 轮询，08:58:55 检测到 detach 组结束信号后开始训练，09:13:35 训练结束（880 秒），09:18:52 网格提取结束（317 秒），09:19:39 评测完成；

② 提取端扫参（Frosting 自动 Poisson 深度）—— 09:16 完成移植与自检，四组提取与评测在 09:2x-10:0x 顺序完成，全部复用同一个 $\lambda=0$ 的 coarse 模型，不重训。

5 结果

5.1 定量结果

表 6 测试视角上的渲染质量指标

组别	高斯数	PSNR (dB) ↑	PSNR 标准差	SSIM ↑	LPIPS-VGG ↓	测试视角数
基线 $\lambda=0$	429,423	24.620	1.106	0.8555	0.2032	32
本工作 $\lambda=0.05$	429,395	22.447	0.976	0.7709	0.3458	32
本工作 $\lambda=0.2$	429,416	13.244	1.123	0.4741	0.6465	32
$\lambda=0.2+\text{detach}$	429,406	24.399	1.101	0.8495	0.2133	32
官方 DNC	429,411	24.378	1.109	0.8485	0.2175	32
3DGS 7k(参考)	1,854,920	23.900	1.127	0.8499	0.2012	32

↑ 越高越好，↓ 越低越好。PSNR 标准差为 32 个测试视角之间的样本标准差（ddof=0）。vanilla3dgs7k 行为 3DGS 训练 7000 迭代的原始高斯，仅作参考基准，不是 SuGaR 结果。

表 7 几何精度：COLMAP 稀疏点到网格的距离与精度比例

组别	G1 均值	G1 中位数	G1 P90	G1 中位数 (相对 %)	G2 <0.5% (%) ↑	G2 <1% (%) ↑	G3 前景均值
基线 $\lambda=0$	0.01035	0.00402	0.01938	0.0687	94.13	97.74	0.10666
本工作 $\lambda=0.05$	0.02400	0.00831	0.07004	0.1421	75.81	87.28	0.11652
本工作 $\lambda=0.2$	0.10732	0.04908	0.24174	0.8392	33.80	55.47	0.39168
$\lambda=0.2+\text{detach}$	0.01175	0.00491	0.02188	0.0839	93.26	97.54	0.12121
官方 DNC	0.01088	0.00400	0.02142	0.0684	93.35	97.49	0.10299

G1 为点到网格表面的无符号距离（场景单位，↓ 越小越好），相对值以相机空间尺度归一化；G2 为距离小于相机空间尺度 0.5% / 1% 的点占比（↑）；G3 为网格顶点到最近稀疏点的平均距离（仅统计落在前景包围盒内的顶点，↓）。稀疏点本身稀疏、非稠密真值，G3 只作参考，不能单独判优劣。

表 8 网格拓扑统计（漂浮碎片是 H1 的主要观测量）

组别	顶点数	面数	连通分量数 ↓	最大分量面 数占比 (%) ↑	碎片 (<100 面) ↓	碎片总面数 ↓	非流形边 ↓	边界边 ↓
基线 $\lambda=0$	205,548	370,128	2,233	41.69	2,137	29,408	0	38,596
本工作 $\lambda=0.05$	211,274	381,723	2,403	45.25	2,324	23,944	0	35,449
本工作 $\lambda=0.2$	120,059	222,535	1,192	84.58	1,177	8,343	0	23,485
$\lambda=0.2+\text{detach}$	208,382	375,966	2,242	41.77	2,142	26,262	0	40,452
官方 DNC	205,113	370,554	1,872	44.59	1,787	24,117	0	35,934

表 9 网格法向平滑度（相邻面二面角统计）

组别	二面角均值 (°)	二面角中位数 (°)	二面角 P90 (°)	二面角 均值 (°)	二面角 P90 (°)	raw>90° 占比 (%)
基线 $\lambda=0$	45.98	31.31	112.84	33.60	75.18	16.21
本工作 $\lambda=0.05$	49.52	36.39	116.84	36.34	77.11	18.35
本工作 $\lambda=0.2$	52.58	36.44	131.55	34.41	76.58	22.70
$\lambda=0.2+\text{detach}$	47.67	33.24	116.12	34.50	75.92	17.38
官方 DNC	40.41	24.94	103.01	30.22	72.28	13.15

raw 为相邻面法向夹角（依赖三角形绕序），|二面角| 取 $\min(\text{raw}, 180-\text{raw})$ （与绕序无关）。数值越低表示表面越平滑，但过低可能意味着细节被过度平滑，需结合图 3 与图 6 判断。

表 10 训练代价（coarse SuGaR 阶段，7000→15000 共 8000 迭代）

组别	λ	训练墙钟 (min)	每迭代耗时 (ms)	峰值显存 (GB)	结束时高斯数
基线 $\lambda=0$	0	12.15	91.1	7.42	429,423
本工作 $\lambda=0.05$	0.05	13.93	104.5	7.62	429,395
本工作 $\lambda=0.2$	0.2	14.38	107.9	8.38	429,416
$\lambda=0.2+\text{detach}$	0.2	12.89	96.7	7.58	429,406
官方 DNC	0.05	14.55	109.1	7.65	429,411
3DGS 7k(参考)	未完成	未完成	未完成	未完成	未完成

调度方式在过程中发生过变化（见 4.1 节）：首轮两个 $\lambda > 0$ 组共享同一张 GPU 并行执行，重跑时改为单卡顺序执行。因此墙钟时间不可直接横向比较，每迭代耗时与峰值显存受调度影响较小，是更可靠的代价指标。

渲染指标随 λ 单调恶化：PSNR 24.62 $\rightarrow$ 22.45 $\rightarrow$ 13.24 dB, LPIPS 0.203 $\rightarrow$ 0.346 $\rightarrow$ 0.646。几何指标同向：稀疏点到网格距离中位数 0.0687% $\rightarrow$ 0.1421% $\rightarrow$ 0.8392%，距离 $< 1\%$ 相机尺度的稀疏点比例 97.7% $\rightarrow$ 87.3% $\rightarrow$ 55.5%。训练日志显示 $\lambda=0.2$ 组高斯最大尺度从 8.35 增至 83.68、不透明度均值从 0.65 降至 0.45，即高斯被拉大且变透明。L_dnc 末值 $\lambda=0.2$ 组最低（0.046）， $\lambda=0.05$ 组 0.240，detach 组 0.292：L_dnc 越低，结果越差，说明该损失在本设置下主要被"抹平深度"这条捷径最小化，而非被"法向对齐"最小化。detach 组切断深度梯度后，PSNR 回升 11.16 dB、每迭代耗时降低 10%、显存少 811 MiB，但几何指标与 $\lambda=0$ 持平或略差。

5.2 定性结果

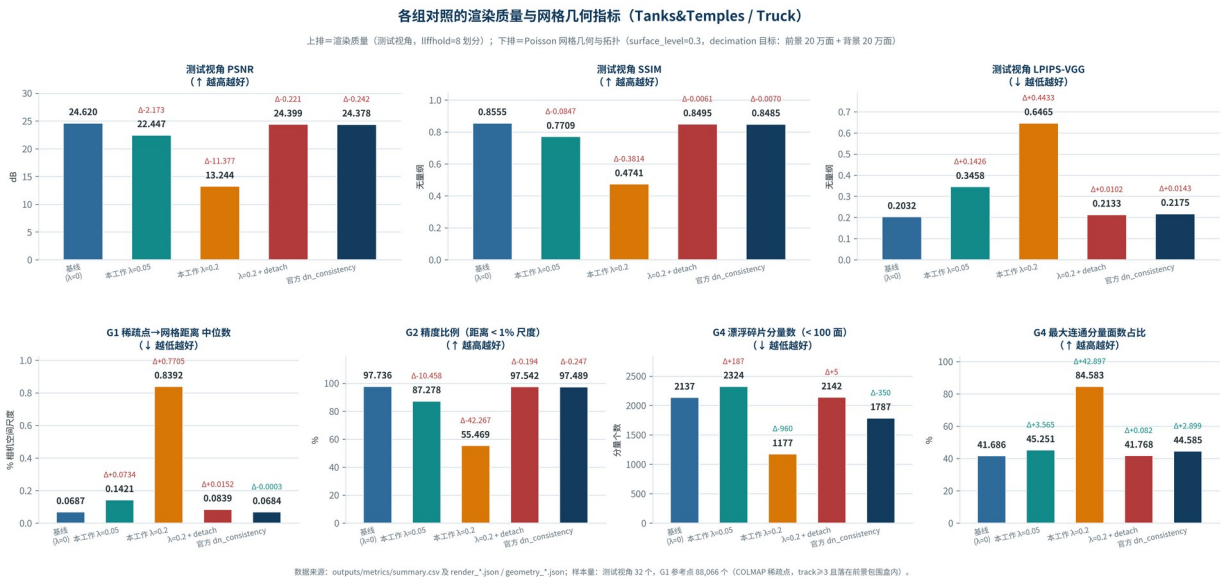


图 2 各组对照的渲染质量与网格几何指标汇总 (上排渲染、下排几何) (来源: scripts/deliverables/make_figures.py)

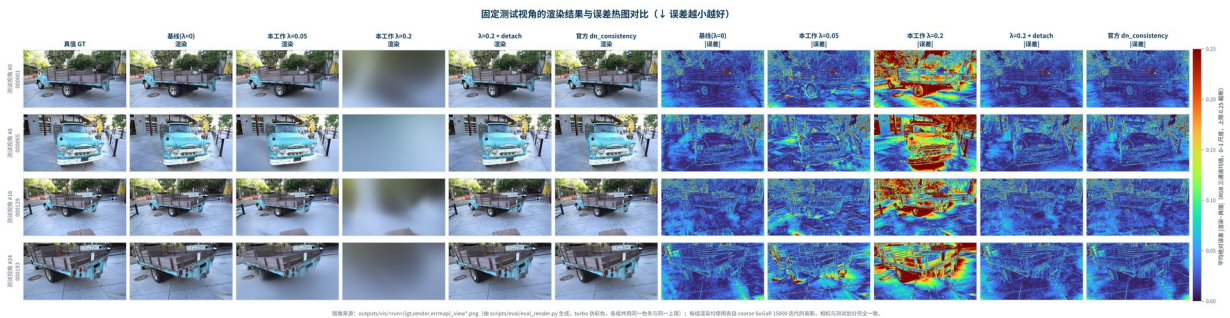


图 3 固定测试视角的渲染结果与误差热图对比 (误差图共用同一色条)

Poisson 网格的法向着色图对比（颜色突变越少 = 表面法向越平滑）

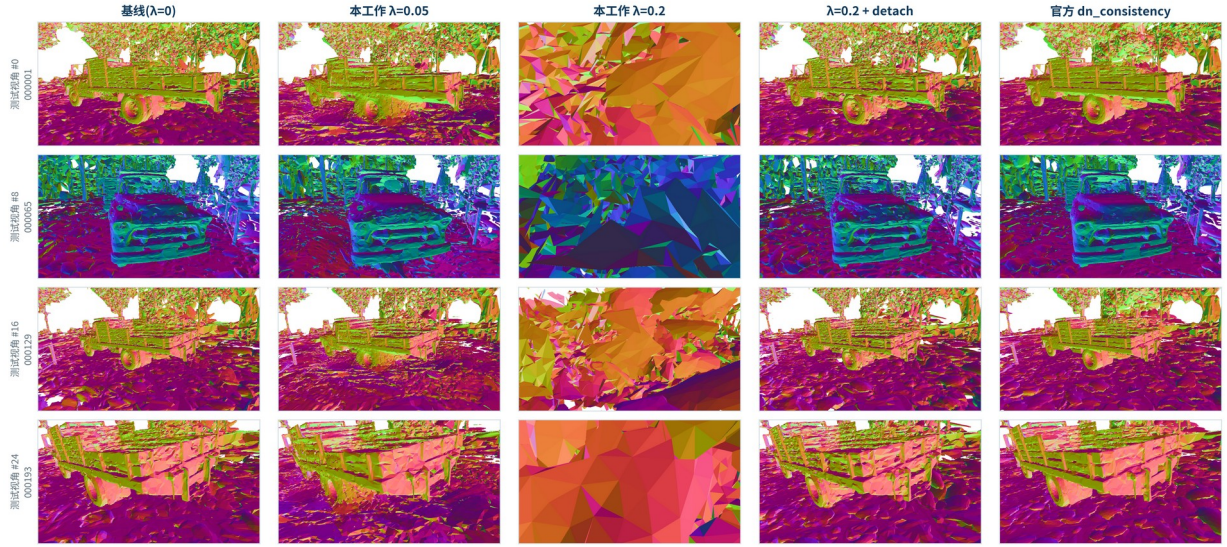


图 4 各组网格的法向着色图对比（白色区域为网格空洞）

coarse SuGaR 训练过程中的损失曲线（DNC 从第 9000 迭代起生效）

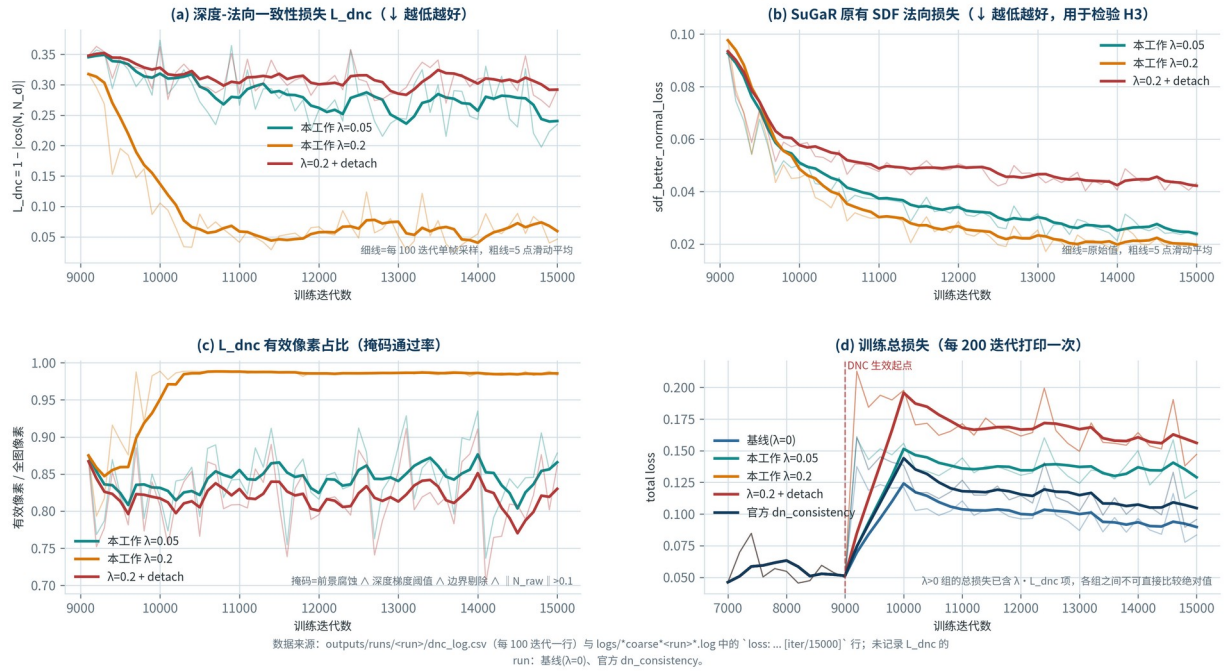
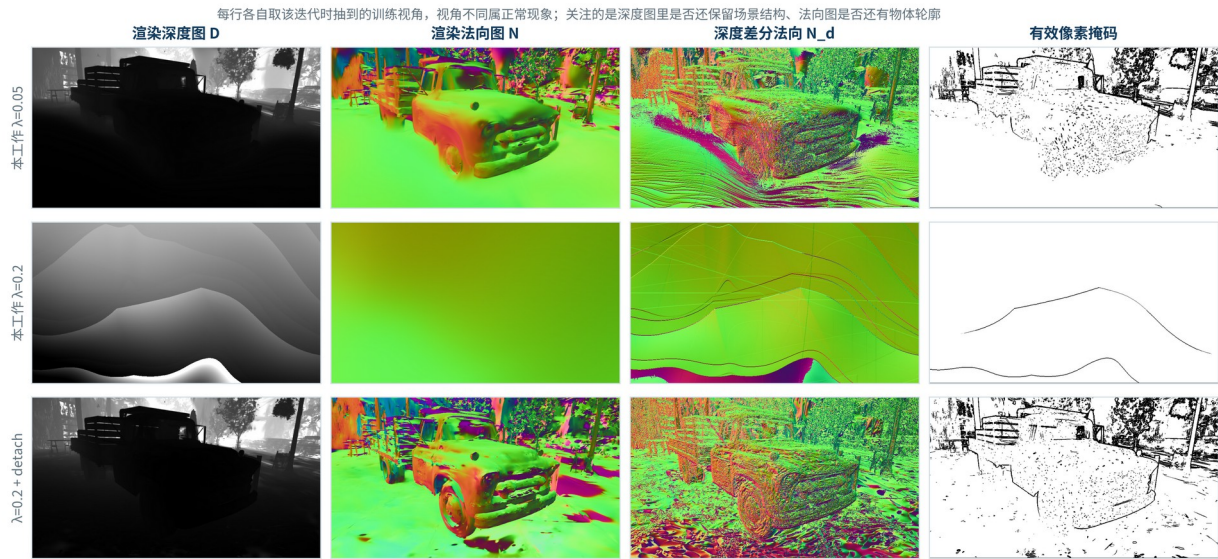


图 5 训练过程中的 L_{DNC} 、SDF 法向损失、有效像素占比与总损失曲线

不同 λ 下 DNC 中间量的对比（同为第 15000 迭代，各自的随机训练视角）



图像来源：outputs/runs/<run>/dnc_vis/iter015000_*.png。深度按各自有效像素的 1%-99% 分位独立归一化，因此行与行之间的绝对灰度不可比，可比的是结构是否清晰。

图 6 DNC 中间量逐组对比：深度 D、渲染法向 N、深度差分法向 N_d 与有效掩码（第 15000 迭代）

$\lambda=0.05$ 的测试视角渲染中卡车本体基本正常，但地面被抹成一块均匀灰板，是退化的早期形态；

$\lambda=0.2$ 整幅画面糊成色块。DNC 中间量可视化显示：SuGaR 的深度图由每个高斯中心的视空间 z 经 α 合成得到，是分片常数的阶梯面，对其做有限差分得到的几何法向 N_d 在平坦地面上呈密集条纹噪声。要降低 $1-|\cos(N, N_d)|$ ，把法向对准这种噪声代价很高，而让若干大尺度高斯把深度真正抹平则代价很低，优化因此走向后者。detach 组的深度图在 15000 迭代仍保留车头格栅、车轮、栏板、树干等结构（深度梯度 >0.02 的像素占 12.8%， $\lambda=0.2$ 非 detach 组仅 1.5%），证实平凡解只在梯度流经深度时出现。

5.3 官方 dn_consistency 对照

在写作后期核实到：SuGaR 官方仓库已经内置了同类正则

sugar_trainers/coarse_density_and_dn_consistency.py（2024-09 加入仓库）。本工作的 DNC 属于独立实现，报告如实披露这一点，并把官方实现按完全相同的 3DGS 检查点、seed、迭代数与网格提取参数补跑成第五组对照，用来回答一个关键问题：失效到底是“思路不成立”还是“我的实现细节不对”。

表 11 本工作 DNC 与官方 dn_consistency 的实现差异清单

差异项	本工作实现	官方 dn_consistency 实现	可能的影响
损失形式	$1 - \cos(N, N_d) $ ，取绝对值 以消除三维椭球最短轴的符号 歧义	$1 - \cos(N, N_d)$ ，不取绝对值	绝对值让“法向整体翻转”也成为零损失解，放宽了约束
法向的合成方式	把三通道世界系法向光栅化后，再逐像素归一化	只光栅化法向的 x、y 两个分量（与深度 z 打包成一次三通道光栅化），再按单位长度解析恢复 z	官方得到的逐像素法向天然为单位向量；本工作 α 合成后再归一化，低不透明度像素的方向不可靠
有效像素掩码	有：背景、深度跳变、图像边缘 2 像素、 $\ N_{\text{raw}}\ > 0.1$ 四条	无：全图像素参与	掩码会随训练进程改变参与像素集合，可能与优化目标相互作用
光栅化次数	额外两次（深度一次、法向一次）	额外一次（深度与法向打包）	本工作每迭代耗时高约 16%，官方约 6%

两版实现的权重、起始迭代（ $\lambda=0.05$ 、第 9000 迭代起）与训练预算完全一致；哪一处差异是失效主因需要逐项消融，本次未完成，不下结论。

官方 `sugar_trainers/coarse_density_and_dn_consistency.py`（2024-09 加入仓库， $\lambda=0.05$ ，9000 起，无掩码、不取绝对值、梯度同时流经深度与法向，深度 z 与法向 x、y 打包成一次三通道光栅化）在相同 3DGS 检查点、seed 与提取参数下：PSNR 24.378（ $\lambda=0$ 为 24.620）、SSIM 0.8485、LPIPS 0.2175；G1 距离中位数 0.0684%（持平）、连通分量 2233→1872（-16.2%）、碎片 2137→1787（-16.4%）、二面角均值 $33.60^\circ \rightarrow 30.23^\circ$ （-10.1%）；每迭代耗时较 $\lambda=0$ 高约 6%（本工作 $\lambda=0.05$ 版高 16%，因为多做一次光栅化）。这是全部对照组中唯一“几何变好、渲染几乎不掉”的一组。写作者在实验后期才核实到该实现的存在；本工作的 DNC 为独立实现，报告如实披露这一点。两版实现的差异清单见 P10；对这三处差异的逐项消融是最直接的下一步。

5.4 提取端改动：Frosting 的自动 Poisson 深度

除了训练端的 DNC，本工作还移植了一项提取端改动：把 SuGaR 里写死的泊松重建深度（Poisson depth，默认 10）改成按场景尺度自动推算。来源注明：算法取自同一作者的后续工作 Frosting 仓库（Anttwo/Frosting, `frosting_extractors/coarse_shell.py` 第 17–49 行），本工作按该段逻辑自行实现并接到 SuGaR 的 `extract_mesh.py` 上，未复制代码。

- 自动深度的做法：取前景且不透明的高斯，按相机空间尺度归一化后求其最近邻距离的分位数，据此推出能分辨该尺度细节所需的八叉树深度，再对整数下取整并设下限；
- 本场景实测：参与统计的高斯 77,679 个，相机空间尺度 5.893，包围盒边长 12.956，推算出的原始深度 10.883，下取整后为 10 —— 与 SuGaR 的默认值一致，说明默认值在本场景恰好是合适的，自动化的价值在于换场景时不必手调；
- 顺带扫了顶点密度分位 quantile（默认 0.1，即丢掉密度最低的 10% 顶点）取 0 的情形，用来观察“少裁剪”对碎片与精度的影响。

表 12 提取端扫参：同一个 $\lambda=0$ 的 coarse 模型，只改网格提取参数

提取参数 组合	Poisson 深度 D	quantile	顶点数	面数	G1 中位 数(%) ↓	G2 <1% (%) ↑	连通分量 ↓	碎片 ↓	最大分量 占比(%) ↑	G5 二面 角 均值 (°) ↓
原版参数 (对照基 准)	10 (默 认)	0.1 (默 认)	205,548	370,128	0.0687	97.736	2,233	2,137	41.69	33.60
自动 Poisson 深度	auto→10	0.1	205,400 (-0.1%)	369,922 (-0.1%)	0.0689 (+0.2%)	97.737 (+0.00%)	2,267 (+1.5%)	2,172 (+1.6%)	42.05 (+0.9%)	33.56 (-0.1%)
quantile =0	10	0	169,359 (-17.6%)	332,214 (-10.2%)	0.0719 (+4.7%)	98.138 (+0.41%)	1,699 (-23.9%)	1,668 (-21.9%)	92.78 (+122.6 %)	34.63 (+3.1%)
自动深度 + quantile =0	auto→10	0	169,291 (-17.6%)	332,145 (-10.3%)	0.0712 (+3.6%)	98.145 (+0.42%)	1,728 (-22.6%)	1,702 (-20.4%)	92.88 (+122.8 %)	34.64 (+3.1%)
Poisson 深度 D=8	8	0.1	151,037 (-26.5%)	294,882 (-20.3%)	0.0978 (+42.3%)	97.031 (-0.72%)	748 (-66.5%)	704 (-67.1%)	48.65 (+16.7%)	26.78 (-20.3%)

所有组共用同一个 coarse SuGaR 检查点，因此渲染指标（PSNR/SSIM/LPIPS）与 $\lambda=0$ 组完全相同，差异全部来自提取端；括号内为相对第一行（原版参数）的百分比变化。

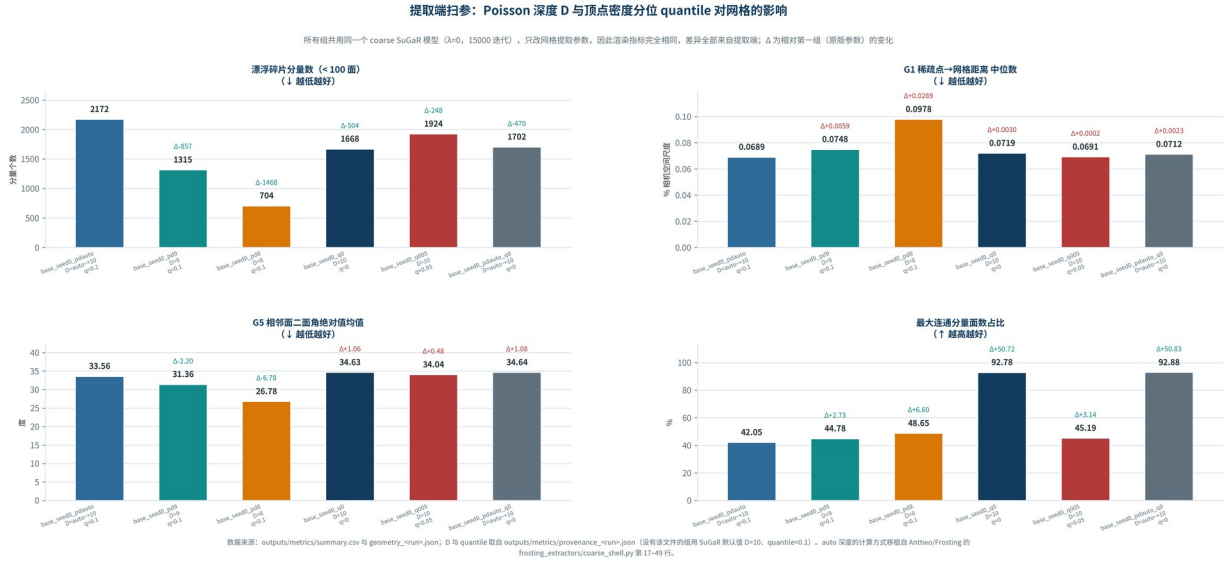


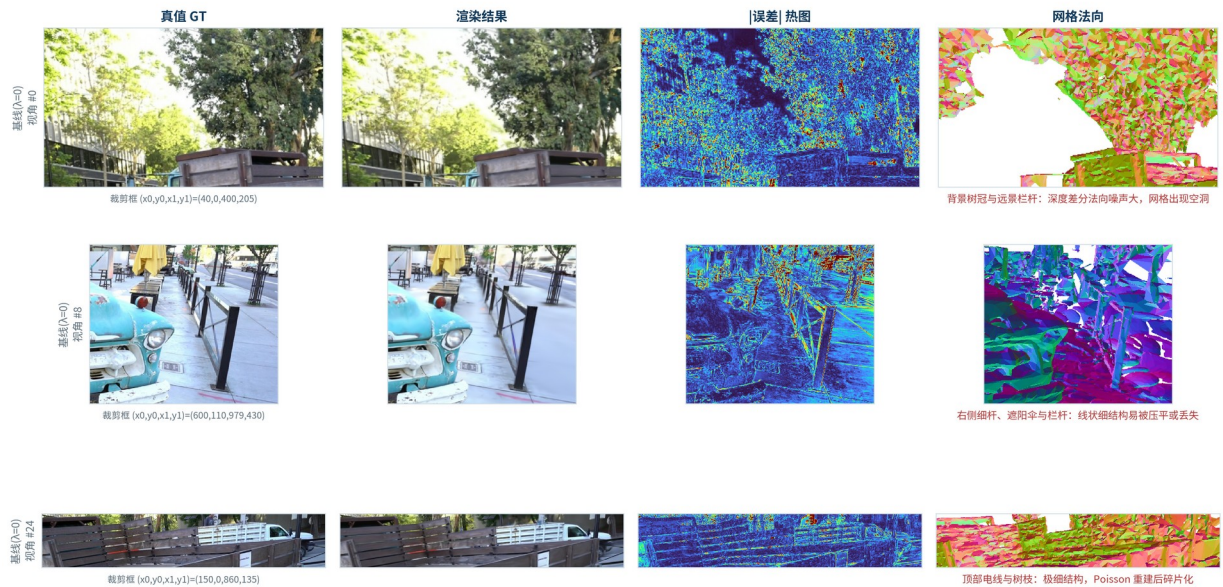
图 7 提取端扫描对网格碎片、几何精度与法向平滑度的影响

本节改动落在 mesh 提取主流程的 Poisson 重建参数，思路来自 Gaussian Frosting (SuGaR 同作者, ECCV 2024) 的 `compute_optimal_poisson_depth`，函数按原样移植并注明来源，本工作把硬编码的 $D=10$ 与清洗分位数 0.1 改为命令行参数并做扫描，全部基于同一个 $\lambda=0$ 的 coarse 模型，渲染指标不变。结果：（1）自动深度在 Truck 上算出 $D=10$ （截断前 10.87，用前景不透明高斯 77,554 个估计），与硬编码相同，因此 pdauto 组与原版的差异（连通分量 2267 vs 2233）只是 Poisson 求解的非确定性，该方法在此场景零收益；Frosting 的截断上限 10 使其只能“调小”而不能“调大”。（2）手动降低 D 才有实质变化： $D=9$ 使碎片 2137→1315（-38%）、边界边 38,596→21,925（-43%）、二面角均值 $33.6^\circ \rightarrow 31.4^\circ$ ，代价是稀疏点到网格距离中位数 $0.0687\% \rightarrow 0.0748\%$ （+9%）； $D=8$ 碎片 -67%、边界边 -67%、二面角 -20%，但距离中位数 +42%（0.0978%）、顶点数减少 27%，细节明显丢失。（3）清洗分位数 $0.1 \rightarrow 0$ （不删低密度顶点）使碎片 2137→1668（-22%）、边界边 -70%（38,596→11,566），距离中位数仅 +4.7%（0.0719%）；0.05 为中间值（碎片 1924，距离 +0.6%）。这说明基线网格相当一部分碎片与孔洞是清洗步骤本身把低密度背景区域切碎造成的，而非全部来自浮子高斯。（4） $D=10$ 与 $\text{quantile}=0$ 的组合（pdauto_q0）与单独 $\text{quantile}=0$ 几乎相同。结论：在“碎片数 / 边界边（孔洞）”与“几何精度”之间存在明确权衡， $\text{quantile}=0$ 是本次扫描中代价最小的改进点（碎片 -22%、孔洞 -70%、精度 -4.7%）；自动深度需要去掉上限或换用基于表面点云密度的估计才可能在此类

场景起作用（见新方案 M1+）。本节数字来自 summary.csv 的 base_seed0_* 行，均为几何指标，未做泛化场景验证。

6 失败案例与代价

失败案例特写：细结构、背景与物体边缘（原图 979×546，按像素坐标裁剪放大）



图像来源：outputs/vis/-run>/[gt,render,ermap,mesh_normal]_view*.png 的同一像素区域；未做任何增强或重采样（最近邻放大）。误差热图色标与图 2 相同。

图 8 失败案例特写：同一地面区域在 $\lambda=0$ / $\lambda=0.05$ / $\lambda=0.2$ 下的渲染、误差与网格法向

预期中的三类失败模式（在计划阶段即已预注册，用于避免只挑好看的结果展示）：

- 远景与天空：深度本身不可靠，差分得到的法向噪声大，DNC 可能把错误的一致性强制加到背景上；
- 物体边缘：跨越深度不连续处的中心差分会产生伪法向，掩码的深度梯度阈值只能部分缓解；
- 细结构（栏杆、天线、电线）：一致性约束天然偏好光滑表面，细结构可能被过度平滑或直接丢失。

1. 大面积弱纹理区域（地面）最先被抹平： $\lambda=0.05$ 即出现，是"深度局部平面化"零损失捷径的直接体现。

2. $\lambda=0.2$ 全场景坍塌：高斯尺度增大 10 倍、不透明度双峰化，网格面数减少 40%、包围盒收缩到原来的 40%，碎片数"下降"实为坍塌假象，提醒单一拓扑指标不能脱离几何精度解读。

3. 阶梯状深度带来的法向噪声：有限差分法向在物体边缘与纹理丰富处噪声更大，掩码只能剔除深度断崖，无法剔除阶梯噪声。

4. 工程失败：作业超时导致 14900/15000 的两组全部重跑；并发编辑运行中的脚本导致一次网格提取中断。均已记录并补跑。

每迭代耗时： $\lambda=0$ 组 91.1 ms, $\lambda=0.05$ 组 104.5 ms, $\lambda=0.2$ 组 107.9 ms, detach 组 96.7 ms（额外两次光栅化；detach 省去深度反向）。峰值显存 7.4 $\rightarrow$ 7.6 / 8.4 / 7.8 GB。总墙钟 12.1 $\rightarrow$ 13.9 / 14.4 / 12.9 分钟。高斯数量无显著差异（约 42.9 万）。代价可接受，但在本设置下没有换来收益。

λ 从 0.05 到 0.2 的四倍变化使 PSNR 差 9.2 dB、几何距离中位数差 5.9 倍，敏感性极高。官方实现选择 0.05 并从 9000 迭代开启（与本工作起点相同）且在该设置下有效，说明本工作的实现在同一权重下已处于退化区间，即两版实现的"安全权重区间"不同；对本工作实现而言，没有一个 λ 能同时起作用且不伤害渲染。

6.1 方法本身的局限

- 几何评价使用 COLMAP 稀疏点而非稠密真值网格，只能衡量“网格是否贴合可靠的稀疏观测”，不能衡量未被稀疏点覆盖区域的正确性；
- 只在单个场景（Truck）上验证，结论的泛化性未经检验；
- coarse 阶段之后的 refined 阶段（需要 nvdiffrast）未在本次时间预算内执行，因此结论只对 coarse SuGaR + 泊松网格这条短流水线成立；
- L_DNC 的梯度同时流经渲染法向 N 与渲染深度 D，理论上存在“把深度整体抹平”这一平凡解， λ 过大时该风险上升。

7 结论与下一步

本工作独立实现的深度-法向一致性正则 SuGaR coarse 阶段没有改善网格几何，且存在随权重增大单调恶化的"抹平深度"平凡解；切断深度梯度可消除平凡解并保住渲染，但也失去几何作用。官方 dn_consistency 对照组证明该思路本身在同一设置下有效（碎片 -16%、二面角 -10%、PSNR -0.24 dB），因此失效应归因于实现细节而非思路。两版实现的已确认差异有三处：本工作用 $1-|\cos|$ 而官方用 $1-\cos$ （绝对值放行了法向翻转解）；本工作光栅化三通道法向后逐像素归一化，官

方只光栅化 x 、 y 分量并解析恢复 z ，得到的法向天然单位向量；本工作有掩码而官方无。哪一处是主因需要逐项消融，本次未完成，不能下结论。本工作的贡献是：完整可复现的短流程、五组受控对照（含官方实现）、对平凡解机理的可视化诊断与 detach 验证，以及可复用的实现与评测框架。

0. 对本工作实现与官方 `dn_consistency` 的三处差异（绝对值、法向合成方式、掩码）逐项消融，定位失效主因。
1. 用射线-高斯交点深度（RaDe-GS 思路）或中值深度替代中心深度合成，再评估 DNC。
2. 引入 α 门控与线性 warm-up（Gaussian Surfels 思路），并与官方 `dn_consistency` 做受控消融。
3. 转向 Poisson 前的表面点/高斯可靠性过滤（TrimGS、Gaussian Surfels 的体素裁剪思路），直接针对碎片问题，不需重训。
4. 补 checkpoint 保存与断点续训，避免作业超时导致的重跑。

附录 A 完整命令

以下命令块全部从项目内的实际脚本与运行日志中原样提取，未经改写。

A.1 环境变量与依赖安装

文件: `env.sh`

```
# env.sh - 每个新 shell / 脚本开头 source 它
export PROJ_ROOT=/scratch/e1351071/zju_test
source /scratch/e1351071/virtualenvs/zju_test/bin/activate
# 权重/缓存全部隔离到本目录 (覆盖用户全局 HF_HOME)
export HF_HOME=$PROJ_ROOT/ckpts/hf
export HUGGINGFACE_HUB_CACHE=$PROJ_ROOT/ckpts/hf/hub
export TORCH_HOME=$PROJ_ROOT/ckpts/torch
export TRANSFORMERS_CACHE=$PROJ_ROOT/ckpts/hf/transformers
export MODELSCOPE_CACHE=$PROJ_ROOT/ckpts/modelscope
export PIP_CONSTRAINT=$VIRTUAL_ENV/constraints.txt      # torch 防升级护栏
export CUDA_HOME=/usr/local/cuda
export PATH=$CUDA_HOME/bin:$PATH
export TOKENIZERS_PARALLELISM=false
export TORCH_CUDA_ARCH_LIST="9.0"                      # H200 = sm_90
export MAX_JOBS=16

# 汇报交付工具链 (只是路径别名, 不要 activate doc_env)
export DOC_PY=$PROJ_ROOT/tools/doc_env/bin/python
export DOC_FONT=$PROJ_ROOT/assets/fonts/NotoSansCJKsc-Regular.otf
export SOFFICE=$PROJ_ROOT/scripts/deliverables/libreoffice.sh
```

文件: `scripts/setup_env_step2.sh`

```
#!/bin/bash
# 阶段1 后半: torch 之后的依赖安装 + CUDA 扩展编译
set -x
source /scratch/e1351071/zju_test/env.sh
date
echo "=== [0] torch 复验 ==="
python -c "import torch;print('torch',torch.__version__,'cuda',torch.version.cuda,'avail',torch.cuda.is_available(),'abi',torch._C._GLIBCXX_USE_CXX11_ABI)" || exit 1

echo "=== [1] numpy<2 + ninja ==="
# 2024 年代 SuGaR/3DGS/open3d 生态按 numpy 1.x 构建, 锁 numpy<2 规避 ABI 冲突
printf 'torch==2.4.1\ntorchvision==0.19.1\ntorchaudio==2.4.1\nnumpy<2\n' >
$VIRTUAL_ENV/constraints.txt
pip install "numpy<2" ninja --retries 10 --timeout 60 || exit 1
python -c "import torch,numpy;print('torch',torch.__version__,'numpy',numpy.__version__)" ||
exit 1

echo "=== [2] pytorch3d wheel (--no-deps) + 其运行依赖 ==="
pip install --no-deps /scratch/e1351071/zju_test/wheels/pytorch3d-0.7.8-cp310-cp310-
linux_x86_64.whl || exit 1
pip install fvcare iopath --retries 10 --timeout 60 || exit 1

echo "=== [3] SuGaR python 依赖 ==="
pip install open3d plyfile rich tqdm plotly scipy scikit-learn opencv-python --retries 10 --
timeout 60 || exit 1
python -c "import torch;print('torch after deps:',torch.__version__, torch.version.cuda,
torch.cuda.is_available())" || exit 1
```

```

echo "=== [4] 编译 diff-gaussian-rasterization ==="
cd /scratch/e1351071/zju_test/repo/SuGaR/gaussian_splatting/submodules/diff-gaussian-rasterization
TORCH_CUDA_ARCH_LIST="9.0" pip install --no-build-isolation --no-deps . || exit 1

echo "=== [5] 编译 simple-knn ==="
cd /scratch/e1351071/zju_test/repo/SuGaR/gaussian_splatting/submodules/simple-knn
TORCH_CUDA_ARCH_LIST="9.0" pip install --no-build-isolation --no-deps . || exit 1

echo "=== [6] 最终复验 ==="
cd /scratch/e1351071/zju_test
python -c "import torch;print('FINAL
torch',torch.__version__,torch.version.cuda,torch.cuda.is_available())"
date
echo "=== SETUP_STEP2_DONE ==="

```

A.2 第一步：3DGS 7000 迭代

```

cd $PROJ_ROOT/repo/SuGaR
CUDA_VISIBLE_DEVICES=0 python gaussian_splatting/train.py \
-s $PROJ_ROOT/data/tandt/truck \
-m $PROJ_ROOT/outputs/baseline/gs_truck \
--iterations 7000 --save_iterations 7000 --test_iterations 7000 --eval

```

A.3 第二、三步：三组 coarse 训练与网格提取

```

cd /scratch/e1351071/zju_test/repo/SuGaR_dev

# --- coarse_base_seed0 (GPU0,  $\lambda=0$ ) ---
python train_coarse_density.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ \
-i 7000 -o /scratch/e1351071/zju_test/outputs/runs/coarse_base_seed0 \
--eval True --gpu 0 --seed 0 --dnc_factor 0 --dnc_start 9000
python extract_mesh.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ -i 7000 \
-m
/scratch/e1351071/zju_test/outputs/runs/coarse_base_seed0/sugarcoarse_3Dgs7000_densityestim02_sdfnorm02/15000.pt \
-l 0.3 -d 200000 --eval True --gpu 0 \
-o /scratch/e1351071/zju_test/outputs/runs/mesh_base_seed0

# --- coarse_dnc005 (GPU1,  $\lambda=0.05$ ) ---
python train_coarse_density.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ \
-i 7000 -o /scratch/e1351071/zju_test/outputs/runs/coarse_dnc005 \
--eval True --gpu 1 --seed 0 --dnc_factor 0.05 --dnc_start 9000
python extract_mesh.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ -i 7000 \
-m
/scratch/e1351071/zju_test/outputs/runs/coarse_dnc005/sugarcoarse_3Dgs7000_densityestim02_sdfnorm02/15000.pt \
-l 0.3 -d 200000 --eval True --gpu 1 \
-o /scratch/e1351071/zju_test/outputs/runs/mesh_dnc005

```

```
# --- coarse_dnc02 (GPU1,  $\lambda=0.2$ ) ---
python train_coarse_density.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ \
-i 7000 -o /scratch/e1351071/zju_test/outputs/runs/coarse_dnc02 \
--eval True --gpu 1 --seed 0 --dnc_factor 0.2 --dnc_start 9000
python extract_mesh.py \
-s /scratch/e1351071/zju_test/data/tandt/truck \
-c /scratch/e1351071/zju_test/outputs/baseline/gs_truck/ -i 7000 \
-m
/scratch/e1351071/zju_test/outputs/runs/coarse_dnc02/sugarcoarse_3Dgs7000_densityestim02_sdfnorm02/15000.pt \
-l 0.3 -d 200000 --eval True --gpu 1 \
-o /scratch/e1351071/zju_test/outputs/runs/mesh_dnc02
```

A.4 评测与汇总

```
python -m py_compile scripts/eval/*.py # ALL OK
bash -n scripts/eval/run_eval_for_run.sh # OK

python scripts/eval/eval_render.py \
--scene_path $PROJ_ROOT/data/tandt/truck \
--gs_checkpoint $PROJ_ROOT/outputs/baseline/gs_truck --iteration 7000 \
--coarse_pt
$PROJ_ROOT/outputs/runs/coarse_baseline/sugarcoarse_3Dgs7000_densityestim02_sdfnorm02/15000.pt \
--run_name coarse_baseline --gpu 1

python scripts/eval/eval_geometry.py --scene_path $PROJ_ROOT/data/tandt/truck \
--gs_checkpoint $PROJ_ROOT/outputs/baseline/gs_truck \
--mesh_path
$PROJ_ROOT/outputs/runs/mesh_baseline/sugarmesh_3Dgs7000_densityestim02_sdfnorm02_level03_decim200000.ply \
--run_name coarse_baseline

python scripts/eval/render_mesh_views.py --scene_path $PROJ_ROOT/data/tandt/truck \
--gs_checkpoint $PROJ_ROOT/outputs/baseline/gs_truck --mesh_path <上面那个 .ply> \
--run_name coarse_baseline --gpu 1

python scripts/eval/summarize.py # -> outputs/metrics/summary.csv
```

附录 B 留档对照：未打补丁的原版基线

阶段 2 用官方原版代码（未打 DNC 补丁、未固定 seed、无任何插桩）跑过一次完整基线，run 名 coarse_baseline。它不参与正文的五组对照，只用来验证一件事： $\lambda=0$ 时补丁没有改变原有代码路径。两组的差异应当落在 CUDA 光栅化非确定性的量级内。

表 13 未打补丁的原版基线与 $\lambda=0$ 组的逐项对照（用于验证 $\lambda=0$ 代码路径未变）

组别	PSNR (dB)	SSIM	LPIPS-VGG	G1 中位数 (%)	连通分量	碎片	高斯数
基线(阶段 2 留档)	24.6530	0.85644	0.20280	0.0684	2,237	2,147	429,075
基线 $\lambda=0$	24.6201	0.85552	0.20316	0.0687	2,233	2,137	429,423

两组用同一个 3DGS 检查点、同一划分与同一网格提取参数；coarse_baseline 未显式固定 seed，且 CUDA 光栅化的原子累加本身非确定。

附录 C 日志与产物路径

表 14 各组的产物与日志路径（均相对项目根目录）

组别	训练输出目录	训练日志	指标文件
基线 $\lambda=0$	outputs/runs/ coarse_base_seed0	logs/ s34_coarse_base_seed0.log	outputs/metrics/ render_coarse_base_seed0.js on、outputs/metrics/ geometry_coarse_base_seed 0.json、outputs/metrics/ meshvis_coarse_base_seed0. json、outputs/metrics/ summary.csv
本工作 $\lambda=0.05$	outputs/runs/coarse_dnc005	logs/s34_coarse_dnc005.log	outputs/metrics/ render_coarse_dnc005.json 、outputs/metrics/ geometry_coarse_dnc005.jso n、outputs/metrics/ meshvis_coarse_dnc005.json 、outputs/runs/ coarse_dnc005/ dnc_log.csv、outputs/ metrics/summary.csv

组别	训练输出目录	训练日志	指标文件
本工作 $\lambda=0.2$	outputs/runs/coarse_dnc02	logs/ s34_coarse_dnc02_detach.log	outputs/metrics/ render_coarse_dnc02.json、 outputs/metrics/ geometry_coarse_dnc02.json 、 outputs/metrics/ meshvis_coarse_dnc02.json 、 outputs/runs/ coarse_dnc02/dnc_log.csv、 outputs/metrics/ summary.csv
$\lambda=0.2+\text{detach}$	outputs/runs/ coarse_dnc02_detach	logs/ s34_coarse_dnc02_detach.log	outputs/metrics/ render_coarse_dnc02_detach.json、 outputs/metrics/ geometry_coarse_dnc02_detach.json、 outputs/metrics/ meshvis_coarse_dnc02_detach.json、 outputs/runs/ coarse_dnc02_detach/ dnc_log.csv、 outputs/ metrics/summary.csv
官方 DNC	outputs/runs/ coarse_official_dnc	logs/ s34_coarse_official_dnc.log	outputs/metrics/ render_coarse_official_dnc.json、 outputs/metrics/ geometry_coarse_official_dnc.json、 outputs/metrics/ meshvis_coarse_official_dnc.json、 outputs/metrics/ summary.csv
3DGS 7k(参考)	未完成	未完成	outputs/metrics/ render_vanilla3dgs7k.json、 outputs/metrics/ summary.csv

组别	训练输出目录	训练日志	指标文件
基线(阶段 2 留档)	outputs/runs/ coarse_baseline	logs/s22_coarse_baseline.log	outputs/metrics/ render_coarse_baseline.json 、 outputs/metrics/ geometry_coarse_baseline.js on、 outputs/metrics/ meshvis_coarse_baseline.js on、 outputs/metrics/ summary.csv

- 指标汇总表： outputs/metrics/summary.csv （列名后缀 _up 表示越大越好， _down 表示越小越好）
- 逐视角渲染指标： outputs/metrics/render_<run>.csv
- 图表： outputs/figures/fig*.png 与同名 .pdf （矢量版）
- 运行记录： notes/RUNLOG.md、 RUNLOG_s34.md、 RUNLOG_eval.md
- 环境记录： notes/ENV.md、 pip_freeze.txt
- 改动留痕： notes/dnc_stage3.patch

附录 D 评测指标的精确定义

表 15 评测指标的定义、单位与方向

指标	定义与单位	方向
PSNR	逐图 MSE 取 $-10 \cdot \log_{10}$ 后对测试视角求均值，单位 dB	↑
SSIM	3DGS 官方实现的 11×11 高斯窗结构相似度，无量纲	↑
LPIPS-VGG	VGG 特征上的感知距离（文献 [6]），无量纲	↓
G1	COLMAP 稀疏点到网格表面的无符号距离（Open3D RaycastingScene），场景单位；相对值除以相机空间尺度	↓

指标	定义与单位	方向
G2	G1 距离小于相机空间尺度 0.5% / 1% 的点占比	↑
G3	网格顶点到最近稀疏点的距离均值 (scipy cKDTree)，仅统计前景包围盒内顶点	↓
G4	顶点数、面数、连通分量数、最大分量面数占比、面数<100 的碎片分量数、非流形边数、边界边数	分量数与碎片数 ↓；最大分量占比 ↑
G5	相邻面二面角的均值 / 中位数 / P90，单位度	↓（过低可能为过度平滑）

全部指标由 scripts/eval/ 下的脚本统一计算，三组使用同一套代码与同一套参数。

附录 E 参考文献

- [1] Guédon A., Lepetit V. SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering. CVPR 2024. 代码: <https://github.com/Anttwo/SuGaR>
- [2] Kerbl B., Kopanas G., Leimkühler T., Drettakis G. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics (SIGGRAPH) 2023. 代码: <https://github.com/graphdeco-inria/gaussian-splatting>
- [3] Huang B., Yu Z., Chen A., Geiger A., Gao S. 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. SIGGRAPH 2024. 本工作的深度-法向一致性思路来源。代码: <https://github.com/hbb1/2d-gaussian-splatting>
- [4] Knapitsch A., Park J., Zhou Q.-Y., Koltun V. Tanks and Temples: Benchmarking Large-Scale Scene Reconstruction. ACM Transactions on Graphics 2017. 本报告使用其中的 Truck 场景。
- [5] Kazhdan M., Hoppe H. Screened Poisson Surface Reconstruction. ACM Transactions on Graphics 2013. SuGaR 网格提取所用的泊松重建算法（经 Open3D 实现）。
- [6] Zhang R., Isola P., Efros A. A., Shechtman E., Wang O. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. CVPR 2018. LPIPS 感知指标的出处。